

Activation Functions

in Neural Networks

A rigorous, intuitive, industry-focused study guide — 2026 edition

The one mental model to keep. A network without activations is just stacked matrix multiplications — and many linear maps collapse into a single linear map that can only draw straight boundaries. The activation is the **nonlinearity** between layers: it bends the space so the network can model curved, complex functions. Almost every trade-off below comes down to two properties:

1. Saturation

When a curve flattens, its derivative $\rightarrow 0$ and gradient information dies on the way back through the network. This is the **vanishing-gradient** problem.

2. Zero-centering

Whether outputs sit around 0. Non-centered outputs force the next layer's weight gradients to share one sign $\rightarrow$ inefficient zig-zag updates.

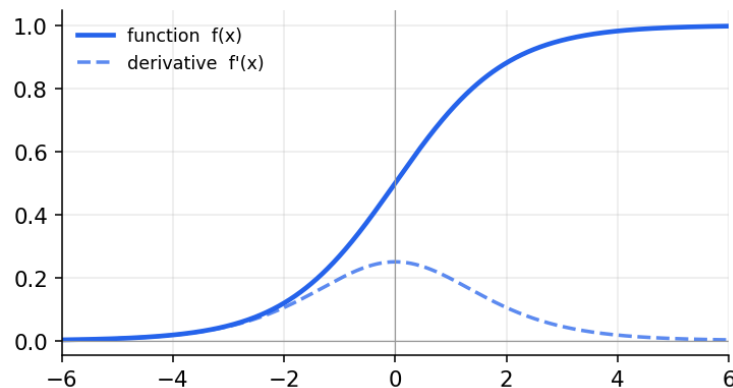
Keep those two in your pocket — they explain almost every advantage and disadvantage in this guide.

1. Sigmoid (Logistic)

The original smooth "soft switch" between 0 and 1

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x) (1 - \sigma(x)) \leq 0.25$$



Output range	(0, 1)
Differentiable	Yes (smooth)
Vanishing risk	High
Compute cost	Medium (exp)
Popularity	Low (output only)

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. Squashes any real number into a smooth S-shape between 0 and 1. Large positive input → near 1 ("on"), large negative → near 0 ("off"). Reads naturally as a probability, which is why it was historically popular.

Advantages

- Smooth, differentiable everywhere
- Bounded output
- Interpretable as a probability

Disadvantages

- Saturates at both tails → vanishing gradients
- Not zero-centered → zig-zag updates
- Uses an expensive exp

Use cases. Output neuron of a binary classifier (gives $P(\text{class}=1)$); gates inside LSTM/GRU cells; multi-label outputs (one independent sigmoid per label). Rare in hidden layers today.

Gradient & backprop. Derivative peaks at just **0.25** (at $x=0$) and decays to 0 in the tails. Multiplying many sub-0.25 numbers through depth shrinks gradients exponentially — the classic vanishing-gradient story.

Efficiency. Moderate: one exp per element.

The curve. Flat low plateau on the left, smooth S-rise through (0, 0.5), flat high plateau on the right.

Interview Qs. Why does sigmoid cause vanishing gradients? (max slope 0.25) | Derive its derivative in terms of σ itself. | Why is 'not zero-centered' a problem?

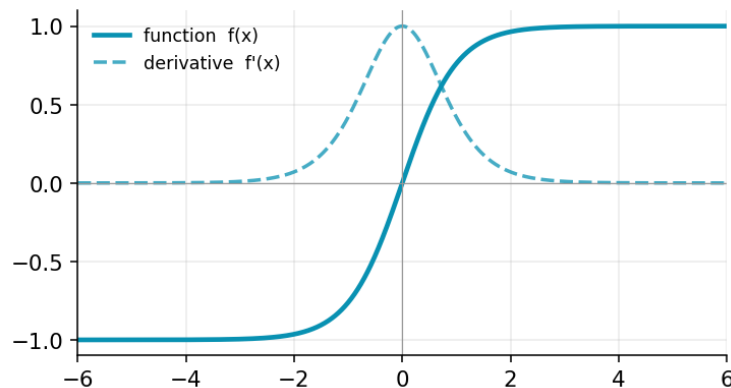
When to choose it. When you need a single probability in (0,1) at the **output** of a binary/multi-label model, or an explicit gate in a recurrent cell. Avoid in deep hidden layers.

2. Tanh (Hyperbolic Tangent)

A zero-centered sigmoid — same shape, signed output

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

$$\tanh'(x) = 1 - \tanh^2(x) \leq 1$$



Output range	(-1, 1)
Differentiable	Yes (smooth)
Vanishing risk	High
Compute cost	Medium (exp)
Popularity	Low-medium (RNNs)

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. A rescaled, zero-centered sigmoid. Same S-shape, but the 'off' state is negative rather than zero. Centering outputs around 0 makes the next layer's learning better-conditioned.

Advantages

- Zero-centered → healthier gradient directions
- Stronger gradients (max slope 1.0 vs 0.25)

Disadvantages

- Still saturates at the tails → vanishing gradients
- Still uses exp

Use cases. Hidden states in classic RNN / LSTM / GRU cells (the candidate cell-state path); shallow networks; settings wanting a bounded, signed output.

Gradient & backprop. Derivative is $1 - \tanh^2(x)$, peaking at **1.0** at $x=0$. Better than sigmoid, but the tails still flatten to 0, so deep stacks still attenuate gradients.

Efficiency. Similar to sigmoid (exponentials), slightly more arithmetic.

The curve. S-shape centered on the origin, passing through (0,0), asymptotes at -1 and +1.

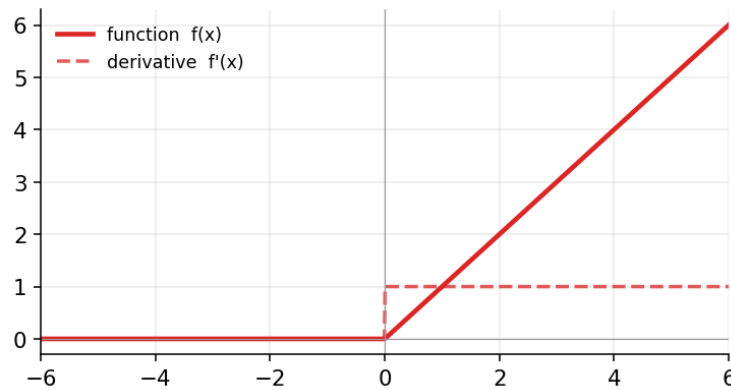
Interview Qs. How does tanh relate to sigmoid? ($\tanh(x)=2\sigma(2x)-1$) | Why prefer tanh over sigmoid for hidden layers?

When to choose it. Inside RNN / LSTM / GRU cells, or shallow MLPs where bounded signed activations help. For deep feedforward/conv nets, prefer the ReLU family.

3. ReLU (Rectified Linear Unit)

The workhorse that made very deep networks trainable

$$\text{ReLU}(x) = \max(0, x)$$



Output range	$[0, \infty)$
Differentiable	No at 0 (subgrad)
Vanishing risk	Low (positive side)
Compute cost	Very low
Popularity	Very high

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. “Pass positives through unchanged, kill negatives.” For positive inputs the derivative is exactly 1, so gradients pass back undiminished — no positive-side saturation. It also yields sparse activations (many exact zeros), which is efficient and mildly regularizing.

Advantages

- Extremely cheap (just a max)
- No positive-side saturation → mitigates vanishing gradients
- Induces sparsity; fast convergence

Disadvantages

- Dying ReLU: a unit can get stuck outputting 0 forever
- Not zero-centered
- Unbounded output can blow up

Use cases. The default hidden activation for CNNs (image classification, detection) and most feedforward nets; the FFN sublayer of many lighter Transformers.

Gradient & backprop. Derivative is 1 for $x > 0$ and 0 for $x < 0$. Clean flow for active units; **zero gradient** for inactive ones — the double-edged sword behind both its success and dying-ReLU failures.

Efficiency. The cheapest of all — a single max. Hardware-friendly.

The curve. Flat zero line for $x < 0$, then a 45° straight ramp for $x > 0$; a sharp elbow at the origin.

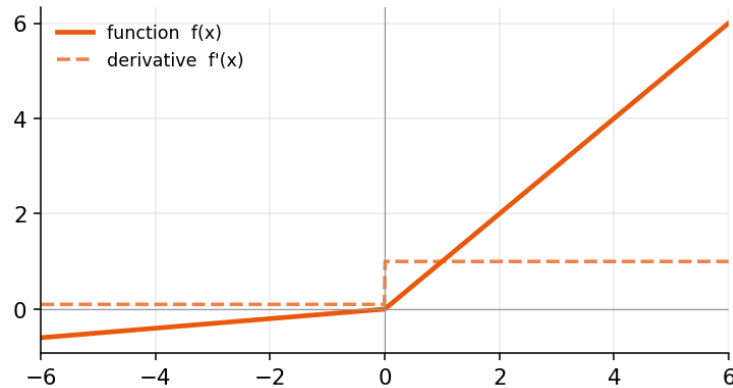
Interview Qs. What is the dying ReLU problem and what causes it? | Why does ReLU help vanishing but not exploding gradients? | Is ReLU differentiable at 0? How is that handled?

When to choose it. Default first choice for hidden layers of CNNs and MLPs when you want speed and simplicity. If many units die, move to Leaky ReLU / ELU / GELU.

4. Leaky ReLU

ReLU with a small leak so neurons never fully die

$$f(x) = x \text{ if } x > 0, \quad \alpha x \text{ if } x \leq 0 \quad (\alpha \approx 0.01)$$



Output range	$(-\infty, \infty)$
Differentiable	No at 0
Vanishing risk	Low
Compute cost	Very low
Popularity	Medium

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. Fix dying ReLU by letting a small gradient leak through on the negative side instead of zeroing it. Dead units can now recover because they always receive some signal.

Advantages

- No dead neurons
- Keeps ReLU's cheapness
- Gradient never fully vanishes on the negative side

Disadvantages

- α is an extra hand-chosen hyperparameter
- Benefit over ReLU is often modest
- Still not zero-centered

Use cases. CNNs prone to dead units; GANs especially (the discriminator commonly uses Leaky ReLU, e.g. DCGAN); anywhere ReLU underperforms from dead units.

Gradient & backprop. Derivative is 1 for $x > 0$ and α for $x < 0$ — small but nonzero everywhere, so no permanently dead path.

Efficiency. As cheap as ReLU (a compare and a multiply).

The curve. Like ReLU but the left half is a gentle downward slope of α instead of flat zero.

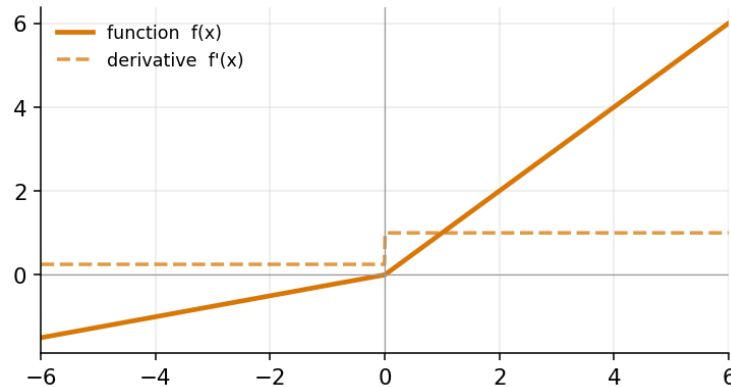
Interview Qs. How does Leaky ReLU solve dying ReLU? | What happens as $\alpha \rightarrow 0$ or $\alpha \rightarrow 1$? (ReLU; linear)

When to choose it. When ReLU shows dead units, or by default in GAN discriminators. A cheap, safe upgrade.

5. PReLU (Parametric ReLU)

Leaky ReLU where the network learns the leak

$$f(x) = x \text{ if } x > 0, \quad \alpha x \text{ if } x \leq 0 \quad (\alpha \text{ learned})$$



Output range	$(-\infty, \infty)$
Differentiable	No at 0
Vanishing risk	Low
Compute cost	Very low (+tiny params)
Popularity	Medium

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. Why guess the negative slope when the network can learn it? PReLU lets each channel decide how 'leaky' it should be. It featured in the first model to beat human-level top-5 accuracy on ImageNet.

Advantages

- Adapts the negative slope to the data
- Can improve accuracy on large vision datasets
- Negligible parameter cost

Disadvantages

- Extra learnable params → mild overfitting risk on small data
- Slightly more complex
- Usually exclude α from weight decay

Use cases. Large-scale image classification where squeezing out accuracy matters and data is plentiful.

Gradient & backprop. Same input-gradient shape as Leaky ReLU, plus a gradient w.r.t. α itself, so the slope is optimized jointly with the weights.

Efficiency. Essentially ReLU cost at inference; training carries a tiny parameter overhead.

The curve. Leaky-ReLU-shaped, but the negative slope is set by training and can differ per channel.

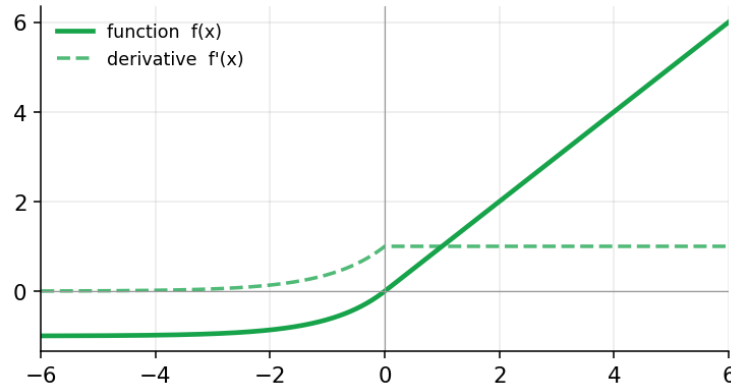
Interview Qs. PReLU vs Leaky ReLU? (learned vs fixed slope) | How many extra params does PReLU add? | Should weight decay apply to α ? (typically no)

When to choose it. Large vision datasets where you want adaptivity and have enough data to avoid overfitting the extra slopes.

6. ELU (Exponential Linear Unit)

Smooth negative side that pushes activations toward zero-mean

$$f(x) = x \text{ if } x > 0, \quad \alpha(e^x - 1) \text{ if } x \leq 0$$



Output range	$(-\alpha, \infty)$
Differentiable	Yes (smooth)
Vanishing risk	Low-medium
Compute cost	Medium (exp)
Popularity	Medium

Solid line = function, dashed line = derivative (the gradient that flows back).

Intuition. Keep ReLU's linear positive side, but make the negative side a smooth curve saturating toward $-\alpha$. This pushes mean activations toward zero (a built-in normalization effect) and gives a smooth, noise-robust negative response.

Advantages

- Negative values push mean activations toward 0 $\rightarrow$ faster, stabler learning
- Smooth everywhere
- No hard dead zone

Disadvantages

- Negative branch uses exp $\rightarrow$ costlier than ReLU
- α to tune
- Far-negative saturation can still attenuate gradients

Use cases. Deeper feedforward/conv nets wanting robustness and self-normalizing behavior without heavy normalization layers. A scaled cousin, SELU, enables self-normalizing networks.

Gradient & backprop. Derivative is 1 for $x > 0$ and αe^x for $x \leq 0$ — smooth, nonzero near the origin, decaying as $x \rightarrow -\infty$.

Efficiency. More expensive than ReLU because of the exponential on the negative side.

The curve. ReLU-like ramp for positives; a smooth curve dipping to a floor of $-\alpha$ for negatives.

Interview Qs. How does ELU push mean activations toward zero, and why help? | ELU vs Leaky ReLU on the negative side? | What is SELU / self-normalization?

When to choose it. Deep nets wanting smoother gradients and near-zero-mean activations, with compute to spare; or SELU for self-normalizing MLPs.

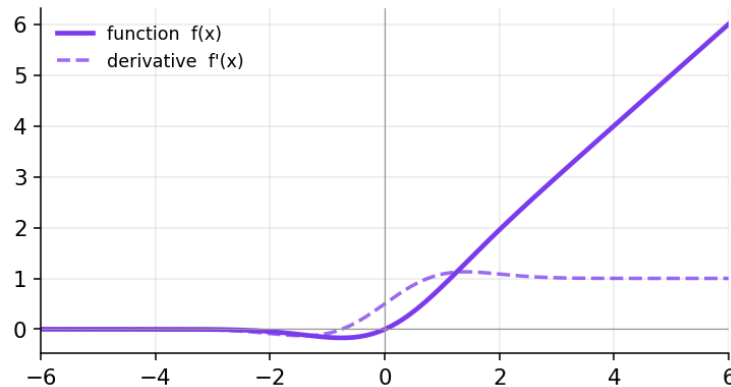
7. GELU (Gaussian Error Linear Unit)

A smooth, probabilistic ReLU — the Transformer default

$$\text{GELU}(x) = x \cdot \Phi(x)$$

$$\approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

$$\text{GELU}'(x) = \Phi(x) + x\phi(x)$$



Solid line = function, dashed line = derivative (the gradient that flows back).

Output range	$\approx (-0.17, \infty)$
Differentiable	Yes (smooth)
Vanishing risk	Low
Compute cost	Medium-high
Popularity	Very high

Intuition. A smooth, probabilistic ReLU. Instead of a hard 0/1 gate, GELU multiplies the input by the probability that a standard Gaussian is below it — small inputs are softly, stochastically down-weighted rather than abruptly killed. The mild non-monotonic dip adds expressivity.

Advantages

- Smooth, differentiable everywhere
- Nonzero gradient for slightly-negative inputs
- Empirically strong in Transformers; extra expressivity

Disadvantages

- More expensive than ReLU (erf / tanh)
- Less intuitive
- Marginal benefit on small/shallow models

Use cases. The feedforward sublayer of Transformers across NLP and vision (BERT, GPT-style LLMs, ViT). Increasingly common in deep CNNs too.

Gradient & backprop. Derivative is $\Phi(x) + x\phi(x)$ — smooth, well-behaved, nonzero on a band of negatives. A key reason it trains deep Transformers stably.

Efficiency. Costlier than ReLU; the tanh approximation is used for speed and is well-optimized on modern accelerators. Negligible vs attention cost.

The curve. A softened ReLU that dips slightly below zero around $x \approx -0.75$ before curving up — smooth, no kink.

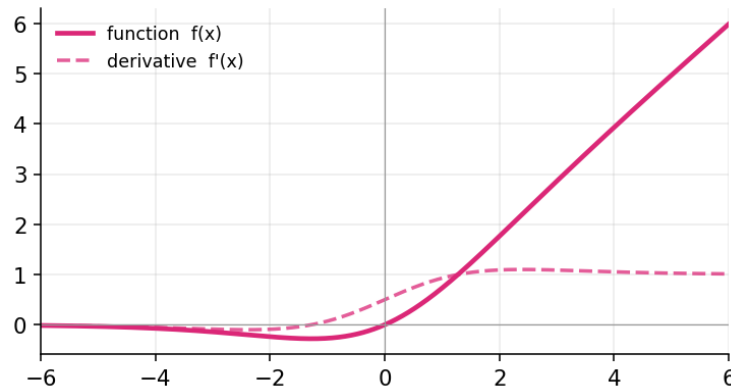
Interview Qs. Why GELU over ReLU in Transformers? | Write GELU via the Gaussian CDF; explain the probabilistic view. | Exact vs tanh approximation — why approximate?

When to choose it. Default for Transformer FFN blocks and large deep models. For tiny/shallow nets the gain over ReLU rarely justifies the cost.

8. Swish / SiLU

A self-gated unit: the input gates itself

$$f(x) = x \cdot \sigma(\beta x) \quad (\text{SiLU when } \beta = 1)$$



Solid line = function, dashed line = derivative (the gradient that flows back).

Output range	$\approx (-0.28, \infty)$
Differentiable	Yes (smooth)
Vanishing risk	Low
Compute cost	Medium-high
Popularity	High

Intuition. A self-gated unit: the input gates itself through a sigmoid. Like GELU it is smooth and non-monotonic (dips slightly negative), letting small negatives carry a little signal. Found by automated search at Google; SiLU ($\beta=1$) is its common form.

Advantages

- Smooth, non-monotonic, differentiable
- Often a small accuracy bump over ReLU in deep nets
- Bounded below, unbounded above

Disadvantages

- More expensive than ReLU (a sigmoid per element)
- Gains can be marginal and model-dependent

Use cases. EfficientNet and many modern CNN backbones; detection/segmentation nets; an alternative to GELU in some Transformers; RL policy/value nets.

Gradient & backprop. Smooth and nonzero across negatives near 0 — similar gradient health to GELU.

Efficiency. Comparable to GELU; pricier than ReLU due to the sigmoid. Hardware-optimized in major frameworks.

The curve. Nearly identical to GELU: a smooth softened-ReLU shape with a small negative trough near $x \approx -1$.

Interview Qs. What does 'self-gated' mean for Swish? | Swish vs GELU — similarities/differences? | What is SiLU? (Swish with $\beta=1$)

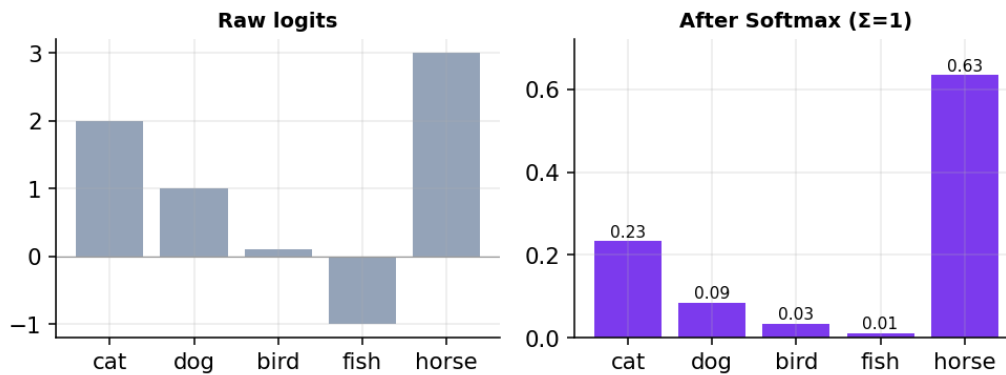
When to choose it. Deep CNN backbones (EfficientNet-style) and places where a smooth ReLU-alternative helps; pick by empirical validation vs GELU/ReLU.

9. Softmax

Turns a score vector into a probability distribution

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$\partial p_i / \partial z_j = p_i (\delta_{ij} - p_j) \Rightarrow \text{with CE: } \hat{p} - y$$



Logits in (left) → a normalized probability distribution out (right). Here 'horse' wins with 0.63.

Output range	(0,1), sums to 1
Differentiable	Yes (smooth)
Vanishing risk	N/A (output)
Compute cost	Medium (exp/class)
Popularity	Very high (output)

Intuition. Unlike the others, softmax acts on a **whole vector**, not one number at a time. It converts arbitrary real scores (logits) into a probability distribution: every output positive, all summing to 1. Exponentiation makes the largest logit dominate (a 'soft argmax') while staying differentiable. Use the log-sum-exp trick (subtract the max logit) for numerical stability.

Advantages

- Interpretable, normalized class probabilities
- Pairs cleanly with cross-entropy → gradient is simply (predicted – true)
- Differentiable

Disadvantages

- Only for mutually-exclusive classes (use sigmoids for multi-label)
- Can be overconfident
- Naive version overflows without the max-subtraction trick

Use cases. The output layer of multi-class classifiers (image-classification heads, language-model next-token distribution) and attention-weight normalization inside Transformers.

Gradient & backprop. Jacobian is $p_i(\delta_{ij} - p_j)$. Combined with cross-entropy it collapses to the clean, stable (predicted – true) gradient — the reason for the softmax+cross-entropy pairing.

Efficiency. One exp per class plus a sum; trivial for normal class counts, heavier for huge vocabularies (motivating hierarchical / sampled softmax).

Interview Qs. Why subtract the max logit first? | Softmax vs sigmoid for outputs? | Derive softmax+cross-entropy gradient. | What is the temperature parameter?

When to choose it. The output layer whenever you need a probability distribution over mutually-exclusive classes, and for normalizing attention scores.

Master Comparison Table

Function	Formula	Output range	Diff?	Vanish risk	Cost	Best use cases	Popularity
Sigmoid	$1/(1+e^{-x})$	(0,1)	Yes	High	Med	Binary/multi-label output, gates	Low
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	(-1,1)	Yes	High	Med	RNN/LSTM hidden states	Low-med
ReLU	$\max(0, x)$	$[0, \infty)$	No@0	Low	V.low	CNNs, MLPs, default hidden	Very high
Leaky ReLU	x or $a \cdot x$	$(-\infty, \infty)$	No@0	Low	V.low	Dead-unit fix, GANs	Medium
PReLU	x or $a \cdot x$ (a learned)	$(-\infty, \infty)$	No@0	Low	V.low	Large-scale vision	Medium
ELU	x or $a(e^x - 1)$	$(-\infty, \infty)$	Yes	Low-med	Med	Deep nets, zero-mean acts	Medium
GELU	$x \cdot \Phi(x)$	$\sim(-0.17, \infty)$	Yes	Low	Med-hi	Transformers (NLP, ViT)	Very high
Swish/SiLU	$x \cdot \sigma(\beta x)$	$\sim(-0.28, \infty)$	Yes	Low	Med-hi	EfficientNet, deep CNNs	High
Softmax	$e^{z_i} / \sum e^{z_j}$	(0,1) sum 1	Yes	n/a	Med	Multi-class output, attention	Very high

The Big “Why” Questions

Why ReLU replaced Sigmoid & Tanh in hidden layers

Sigmoid and tanh saturate — their derivatives collapse toward zero in the tails (sigmoid's peaks at just 0.25). Backprop multiplies these small numbers layer after layer, so gradients reaching early layers shrink exponentially and they barely learn. ReLU's positive-side derivative is exactly 1, so gradients pass back undiminished, making 10–100+ layer networks trainable. ReLU is also far cheaper and sparse. Better gradient flow + lower cost + sparsity = the shift that helped ignite deep learning (AlexNet, 2012).

Why GELU is used in Transformers

Transformers are very deep and trained on enormous data, where small per-layer gains compound. GELU is a smooth, soft-gating alternative to ReLU: instead of hard-zeroing negatives it weights each input by the probability mass below it, keeping a nonzero gradient for slightly-negative inputs and adding a mild non-monotonic dip that boosts expressivity. Empirically this trains large language and vision Transformers slightly more stably and to better accuracy, and the extra compute is negligible against attention. Hence BERT, the GPT family, and ViT standardized on it.

Why Softmax is used in the output layer for classification

Classification needs a probability distribution over mutually-exclusive classes — values in (0,1) summing to 1. Softmax delivers exactly that with a differentiable soft-argmax. Paired with cross-entropy, its gradient simplifies to the clean, stable (predicted – true), which optimizes beautifully. For independent labels (multi-label) use one sigmoid per class instead, because softmax forces the classes to compete.

How activations affect training stability & convergence

Gradient magnitude: saturating functions cause vanishing gradients; unbounded ones risk exploding activations without normalization. Zero-centering: non-centered outputs bias the next layer's gradients to one sign (zig-zag). Smoothness: smooth functions give continuous gradients that suit adaptive optimizers and can lift final accuracy. Sparsity/dead units: ReLU's exact zeros aid efficiency but risk dead neurons. And activation choice is coupled with normalization (BatchNorm/LayerNorm) and initialization (He init for ReLU-family) — the combination is what keeps activation statistics stable across depth and makes deep training converge.

Practical Examples by Task

Image classification (CNN, e.g. ResNet on ImageNet)

Hidden layers: ReLU by default (cheap, fast); Swish/SiLU in EfficientNet-style backbones for a small accuracy gain.

Output layer: Softmax over the mutually-exclusive classes + cross-entropy.

Analogy: ReLU is an assembly-line inspector passing good parts straight through and discarding defects instantly; softmax is the final judge splitting 100% confidence across labels.

NLP (Transformer, e.g. a GPT-style LLM)

Hidden layers: GELU in the feedforward sublayers (or a gated SwiGLU variant in current LLMs).

Output layer: Softmax for attention weights and for the final next-token distribution over the vocabulary.

Analogy: GELU is a dimmer switch (soft, graded) vs ReLU's on/off switch; attention softmax is a budget of 100% attention split among the words in context.

Regression (e.g. predicting house price or temperature)

Hidden layers: ReLU / GELU / Swish.

Output layer: No activation (linear/identity), because the target is an unbounded real number. Use sigmoid if the target is bounded in $[0,1]$; softplus/ReLU if strictly non-negative.

Analogy: The output activation is dictated by the target's range; hidden activations are chosen for gradient health and cost.

Decision Tree — Choosing an Activation

START: Which layer are you choosing for?

```
|-- OUTPUT layer? -> decide by the TARGET:
|   |-- Multi-class, one label per example -> SOFTMAX (+ cross-entropy)
|   |-- Binary classification (one yes/no) -> SIGMOID (+ BCE)
|   |-- Multi-label (independent yes/no) -> SIGMOID per output (+ BCE)
|   |-- Regression (real-valued target) -> LINEAR / none
|       (SIGMOID if bounded [0,1]; SOFTPLUS/ReLU if >= 0)
|
|-- HIDDEN layer? -> decide by ARCHITECTURE:
|   |-- Transformer (NLP / Vision)? -> GELU (or gated SwiGLU/GeGLU)
|   |-- Deep CNN, accuracy-critical? -> SWISH / SiLU (else ReLU)
|   |-- RNN / LSTM / GRU cell? -> TANH (state) + SIGMOID (gates)
|   |-- Standard CNN / MLP? -> ReLU (the default)
|       |-- many dead units? -> LEAKY ReLU
|       |-- lots of data? -> PReLU
|       |-- want zero-mean? -> ELU
|   |-- Building a GAN? -> LEAKY ReLU (discriminator); TANH (generator out)
```

Rule of thumb: hidden layers → start with ReLU, upgrade to GELU/Swish for deep or Transformer models, or Leaky/ELU if you see dead units. Output layer → let the target's range decide (Softmax / Sigmoid / Linear).

Python Implementations & Derivatives

NumPy — forward passes

```
import numpy as np
from scipy.special import erf

def sigmoid(x):      return 1/(1+np.exp(-x))
def tanh(x):         return np.tanh(x)
def relu(x):         return np.maximum(0.0, x)
def leaky_relu(x, a=0.01): return np.where(x>0, x, a*x)
def prelu(x, a):     return np.where(x>0, x, a*x) # a is learned
def elu(x, a=1.0):   return np.where(x>0, x, a*(np.exp(x)-1))
def gelu(x):         return 0.5*x*(1 + erf(x/np.sqrt(2))) # exact
def swish(x, b=1.0): return x*sigmoid(b*x)             # SiLU when b=1

def softmax(z, axis=-1): # numerically stable
    z = z - np.max(z, axis=axis, keepdims=True) # log-sum-exp trick
    e = np.exp(z)
    return e / np.sum(e, axis=axis, keepdims=True)
```

NumPy — derivatives

```
# ---- analytical derivatives (verified vs numerical grad to ~1e-10) ----
def d_sigmoid(x): s=sigmoid(x); return s*(1-s) # <= 0.25
def d_tanh(x):    return 1 - np.tanh(x)**2      # <= 1
def d_relu(x):    return (x>0).astype(x.dtype)  # subgrad 0 at 0
def d_leaky(x,a=0.01): return np.where(x>0, 1.0, a)
def d_elu(x,a=1.0): return np.where(x>0, 1.0, a*np.exp(x))
def d_gelu(x):
    Phi = 0.5*(1+erf(x/np.sqrt(2)))
    phi = np.exp(-x**2/2)/np.sqrt(2*np.pi)
    return Phi + x*phi # Phi(x)+x*phi(x)
def d_swish(x,b=1.0): s=sigmoid(b*x); return s + b*x*s*(1-s)
```

PyTorch — built-ins, a model, and a learnable PReLU

```
import torch, torch.nn as nn, torch.nn.functional as F

torch.sigmoid(x); torch.tanh(x); F.relu(x)
F.leaky_relu(x, negative_slope=0.01); F.elu(x, alpha=1.0)
F.gelu(x) # exact; F.gelu(x, approximate='tanh') = fast form
F.silu(x) # SiLU / Swish (beta = 1)
F.softmax(logits, dim=-1)

net = nn.Sequential(
    nn.Linear(784, 256), nn.GELU(),
    nn.Linear(256, 128), nn.SiLU(),
    nn.Linear(128, 10)) # output RAW LOGITS
prelu = nn.PReLU(num_parameters=128) # learnable slope, init 0.25

# nn.CrossEntropyLoss = LogSoftmax + NLLLoss, fused for stability.
# So the final layer outputs logits -- do NOT add softmax yourself.
```

Why CrossEntropyLoss already contains softmax: it fuses LogSoftmax + NLLLoss, which is more numerically stable (works in log-space, avoids overflow) and yields the clean (predicted – true) gradient. Adding your own softmax before it double-normalizes and hurts training — a very common bug.

Current Best Practices (2026)

■ Hidden default is no longer just ReLU for large models

Plain ReLU is still right for small / edge / latency-critical CNNs and MLPs, but large deep models default to smooth activations — GELU and SiLU/Swish.

■ Gated activations dominate large language models

State-of-the-art LLMs predominantly use gated-linear-unit variants in the feedforward block — SwiGLU (Swish-gated) and GeGLU (GELU-gated). They reliably beat a plain GELU FFN at scale, at the cost of an extra weight matrix (often offset by a narrower hidden width).

■ Vision Transformers and modern CNN backbones

ViTs lean on GELU; EfficientNet-style CNNs on SiLU; ConvNeXt-style modern CNNs adopt GELU to mirror Transformer design.

■ Output layers are settled

Softmax (mutually-exclusive multi-class), Sigmoid (binary/multi-label), linear (regression). This has not moved and is unlikely to.

■ Always co-design activation + normalization + init

He/Kaiming init with the ReLU family; pair smooth activations with LayerNorm (Transformers) or BatchNorm (CNNs). The combination — not the activation alone — is what keeps activation statistics stable across depth and drives convergence.

■ Don't over-optimize the activation early

Differences among GELU/Swish/ELU are usually a fraction of a percent. Get data, architecture, normalization, and learning-rate schedule right first; treat the exact smooth activation as a late-stage knob, validated empirically.

One-line defaults cheat-sheet

Scenario	Pick
CNN (small / fast / edge)	ReLU
CNN (accuracy-critical)	SiLU / Swish
Transformer / LLM	GELU or SwiGLU
RNN / LSTM / GRU	tanh + sigmoid
GAN discriminator	Leaky ReLU
Output: multi-class	Softmax
Output: binary / multi-label	Sigmoid
Output: regression	Linear / none

Note: the mathematical behavior of these functions is fixed and timeless; the 'what's fashionable at scale' items reflect the trajectory into early 2026 and evolve fastest in the LLM space — verify against current papers for frontier work.